

Ensemble Methods

ECMM422

Fabrizio Costa

Winter 2026



Outline

1 Decision Trees

- Partitioning feature space for prediction
- Regression trees
- Classification Trees

2 Bagging

- Random Forests

3 Boosting

- AdaBoost



Space partitioning

- ▶ Let's assume a partition $\mathcal{R}$ of the feature space $\mathcal{X}$ into M regions $R_1, R_2, \dots, R_M$
- ▶ Let's model the predicted response in an entire region R_m simply as a constant c_m
- ▶ The overall predictive function can be expressed as:

$$\hat{f}(x) = \sum_{m=1}^M c_m \mathbb{I}[x \in R_m]$$

- ▶ where $\mathbb{I}[x]$ is the **indicator function**, which is 1 if the argument is true and 0 otherwise

- ▶ How can we choose the best constant value c_m in each part?
- ▶ If we adopt as the objective to minimise the sum of squares

$$\sum (y_i - \hat{f}(x_i))^2$$

- ▶ then the best c_m is obtained by averaging:

$$c_m = \langle y_i \rangle_{x_i \in R_m}$$

- ▶ Here with $\langle y_i \rangle_{x_i \in R_m}$ we indicate the average of all values y_i associated to x_i in the region R_m

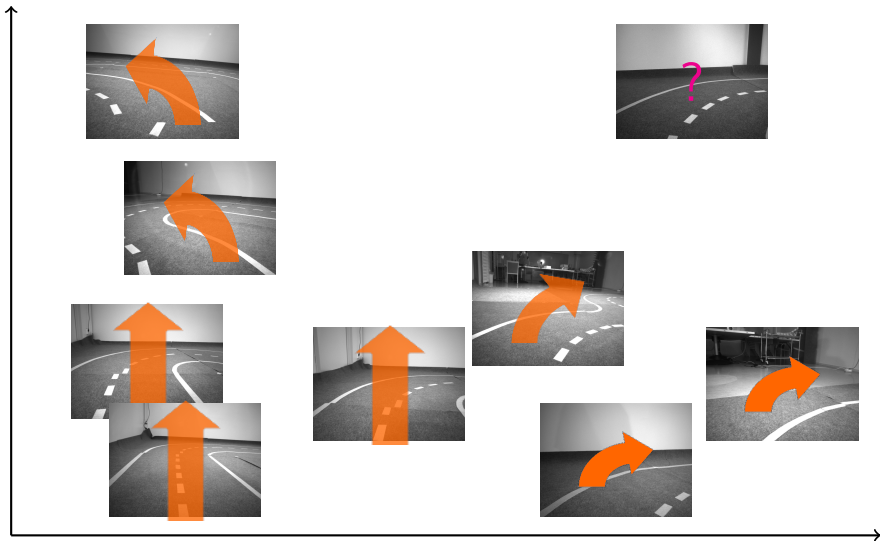
- ▶ Finding the optimal partitioning $\mathcal{R}$ is in general not tractable
- ▶ In practice, we can try and find a *good partition* by **greedy recursive** partitioning of the feature space.

Feature space (X)



Feature space (X)

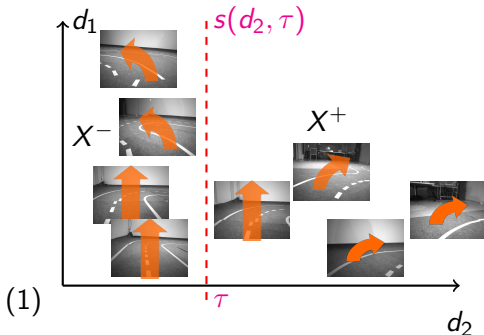




Space partitioning

- ▶ Images are described by n -dimensional instances $x = (x_1, x_2, \dots, x_n) \in X$
- ▶ We define a partition $s(d, \tau) : X \rightarrow (X^+, X^-)$, as:

$$\begin{cases} X^+ := \{x : x_d \geq \tau\} \\ X^- := \{x : x_d < \tau\} \end{cases}$$



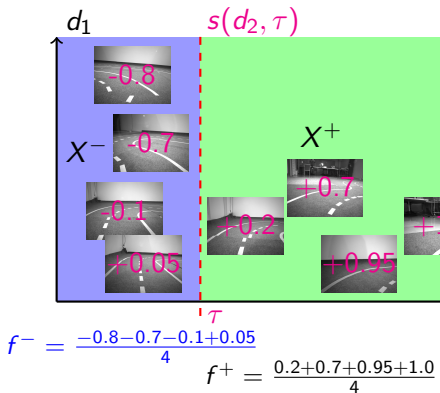
A very weak predictor

- We estimate steering as the expected value for both partitions:

$$\begin{cases} \hat{f}^+ = \langle y(x) \rangle_{X^+} \\ \hat{f}^- = \langle y(x) \rangle_{X^-} \end{cases} \quad (2)$$

- we choose the split s that minimises prediction error:

$$\varepsilon = \sum_{x \in X^+} (\hat{f}^+ - y(x))^2 + \sum_{x \in X^-} (\hat{f}^- - y(x))^2 \quad (3)$$



Decision trees

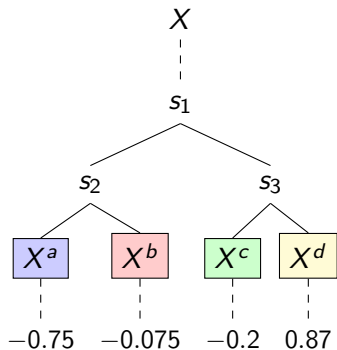


Figure: A decision tree.

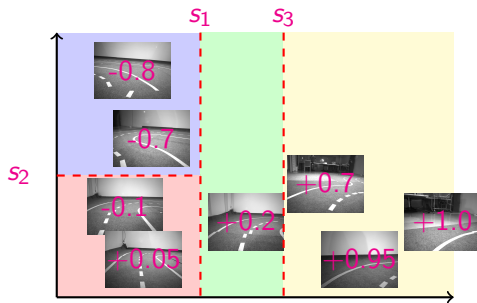
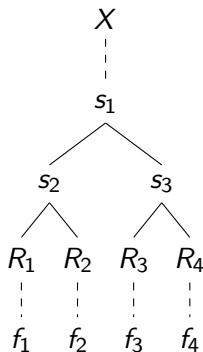


Figure: Feature space partition.

Formally: Decision trees

- ▶ A decision tree is a **binary tree**, where each non-leaf node represents a **binary partition** of the feature space.
- ▶ Each leaf node defines a partition $R_1, \dots, R_m$ of the feature space $\mathcal{X}$.
 - ▶ i.e. each leaf node m corresponds to a region of the feature space R_m
- ▶ Each region contains N_m training examples
- ▶ The predicted value in the region is $\hat{f}_m$
- ▶ The prediction error is

$$\varepsilon(R_m) = \sum_{x \in R_m} (y(x) - \hat{f}_m)^2$$



A simple tree prediction algorithm

```
function PREDICTNODE( $x$ , TreeNode)
  if TreeNode is a leaf node then
    return  $\hat{f}(\text{TreeNode})$ 
  else
    get a split  $(d, \tau) \leftarrow s(\text{TreeNode})$ 
    if  $x_d < \tau$  then
      return PREDICTNODE( $x$ , leftChild(TreeNode))
    else
      return PREDICTNODE( $x$ , rightChild(TreeNode))
    end if
  end if
end function
```

Finding the best partition

We consider a split defined over a region $R \subseteq \mathcal{X}$ as the tuple $s = (d, \tau)$ where d is a feature and a τ is a threshold, such that:

$$R_s^- = \{x \in R \mid x_d \leq \tau\} \quad (4)$$

$$R_s^+ = \{x \in R \mid x_d > \tau\} \quad (5)$$

Then the partition error is given by:

$$\varepsilon(R, s) = \min_{\hat{f}^-} \sum_{x \in R_s^-} (y(x) - \hat{f}^-)^2 + \min_{\hat{f}^+} \sum_{x \in R_s^+} (y(x) - \hat{f}^+)^2 \quad (6)$$

where the minimisation yields for $\hat{f}^-$ the average response in R_s^- and for $\hat{f}^+$ the average response in R_s^+

Then the best split is given by minimising this error:

$$s^*(R) = \underset{s}{\operatorname{argmin}} \varepsilon(R, s) \quad (7)$$

Learning a decision tree from data

- ▶ Idea: Split the feature space recursively, learning each sub-tree from a sub-region of the feature space.
- ▶ This is a **greedy search** approach: each step attempts to maximise reward.
- ▶ How large should we grow the tree?
A very large tree might overfit the data, while a small tree might not capture the important structure.

- ▶ Termination criteria:
 - ▶ All samples have the same value;
 - ▶ Too few samples to split further ($N_m < \kappa$);
 - ▶ Error in region is small or null $\varepsilon(R_m) = 0$;
 - ▶ Max depth η has been reached;
- ▶ Note that lack of improvement is **not** a termination criterion: a mediocre partition can open the way to a good one later on

A simple tree learning algorithm

```
function TRAINNODE( $X^m$ , depth)
  if  $|X^m| > \kappa$  and depth  $< \eta$  and  $\text{Var}(y(X^m)) > 0$  then
    find the best split  $s \leftarrow \arg \min_{d,\tau} \varepsilon(X^m, s_{d,\tau})$ 
    split the dataset  $X^m$  according to  $s$ , into  $X^+$  and  $X^-$ 
    leftChild  $\leftarrow$  TRAINNODE( $X^-$ , depth+1)
    rightChild  $\leftarrow$  TRAINNODE( $X^+$ , depth+1)
    return NewSplitNode( $s$ )
  else
    calculate partition prediction  $\hat{f}_m$ 
    return NewLeafNode( $\hat{f}_m$ )
  end if
end function
```

Decision trees for classification

- ▶ Targets are categorical: $y(x) \in \{\lambda_1, \lambda_2, \dots, \lambda_C\}$
- ▶ Let $\hat{p}_{m,\lambda}$ be the ratio of examples of class λ in a node m representing a part R_m with N_m observations,

$$\hat{p}_{m,\lambda} = \frac{1}{N_m} \sum_{x \in R_m} \mathbb{I}[y(x) = \lambda] \quad (8)$$

- ▶ We classify the samples in node m to class $\hat{f}_m = \operatorname{argmax}_{\lambda} \hat{p}_{m,\lambda}$
- ▶ I.e. the estimation function becomes a majority vote (i.e. the mode)

Objective functions for classification

We can define several objectives as measures of node impurity:

Misclassification error

$$\varepsilon_M(R_m) = \frac{1}{N_m} \sum_{\mathbf{x} \in R_m} \mathbb{I} \left[y(\mathbf{x}) \neq \hat{f}_m \right] = 1 - \hat{p}_{m, \hat{f}_m} \quad (9)$$

Gini index

$$\varepsilon_G(R_m) = \sum_{\lambda} \sum_{\lambda' \neq \lambda} \hat{p}_{m, \lambda} \hat{p}_{m, \lambda'} = \sum_{\lambda} \hat{p}_{m, \lambda} (1 - \hat{p}_{m, \lambda}) \quad (10)$$

Entropy

$$\varepsilon_E(R_m) = - \sum_{\lambda} \hat{p}_{m, \lambda} \log \hat{p}_{m, \lambda} \quad (11)$$

- ▶ Is a measure of how often a randomly chosen element from the set would be incorrectly labeled if it were randomly labeled according to the distribution of labels in the subset
- ▶ It can be computed by summing the probability of each item being chosen times the probability of being mistaken in choosing that label for categorizing that item
- ▶ The minimum value of the Gini index is 0. This is obtained when the node contains element of a single class (no need for further splits)
- ▶ The optimum split is chosen by the features with the smallest Gini Index
- ▶ The Gini index is maximum when the probability of the classes are the same

Entropy

- ▶ **Entropy** is a measure of the uncertainty associated with a random variable X
- ▶ It quantifies the expected value of the **information** contained in a message
- ▶ The information content should be defined in a way such that it is true that:
 - ▶ An event that is certain is perfectly unsurprising and yields no information
 - ▶ The less probable an event is, the more surprising it is and the more information it yields.
 - ▶ If two independent events are measured separately, the total amount of information is the sum of the self-informations of the individual events.
- ▶ Hence the information content of X must have the form:

$$I(X) = -\log p(X)$$

- ▶ The entropy H of a random discrete variable X with possible values $\{x_1, \dots, x_n\}$ and probability function $p(X)$, is formally defined as:

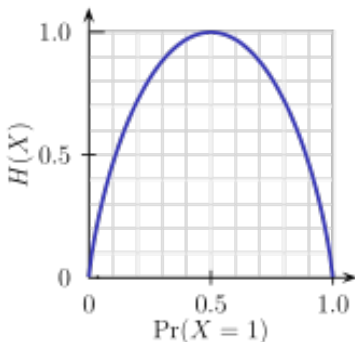
$$H(X) = E[I(X)] = E[-\log p(X)]$$

- ▶ hence:

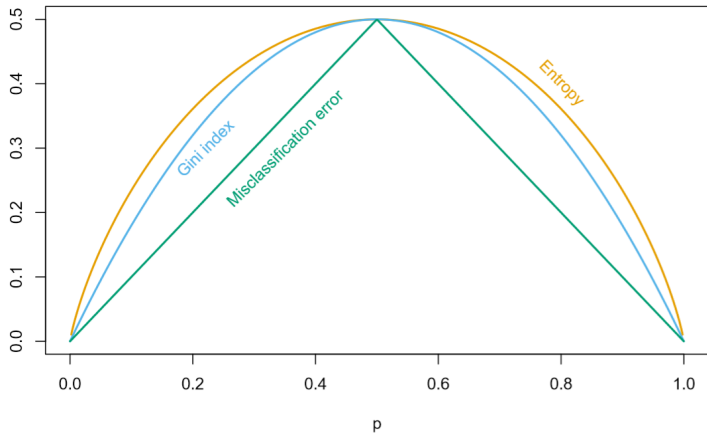
$$H(X) = \sum_{i=1}^n p(x_i) I(x_i) = - \sum_{i=1}^n p(x_i) \log p(x_i)$$

Entropy

- ▶ The entropy can be thought as the **expected surprise**
- ▶ Consider a coin flip where only two outcomes are possible
- ▶ Say that the random variable $X = 1$ represents a result of heads
- ▶ Then the entropy of X is the *number of bits* that we'll need in order to communicate the outcome of the coin flip w.r.t. the fairness of the coin

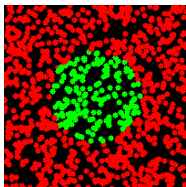


Objective functions for classification

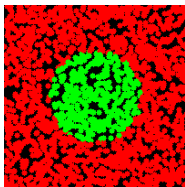


Node impurity measures for two-class classification, as a function of the proportion p in class 2.

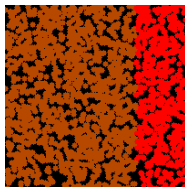
Effect of depth



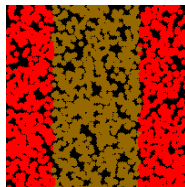
training data



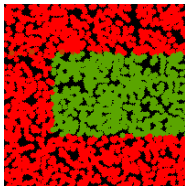
ground truth



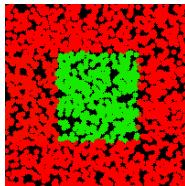
$\eta = 1$



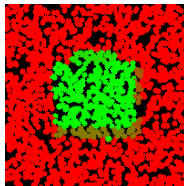
$\eta = 2$



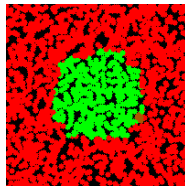
$\eta = 3$



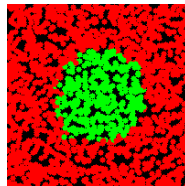
$\eta = 4$



$\eta = 5$

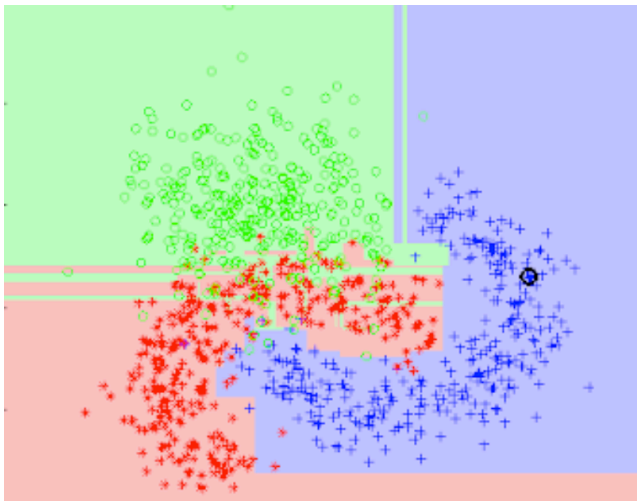


$\eta = 10$



$\eta = 15$

Decision Tree overfitting



Overfitting: tree pruning

- ▶ Once a maximal depth tree T_0 is learnt, it is possible to prune it, that is, to collapse any number of its internal (non-terminal) nodes.
- ▶ In practice, it is done by minimising a criterion that includes a penalty term for the complexity of the tree:

$$C_\alpha(T) = \sum_{m=1}^{|T|} \varepsilon(R_m) + \alpha|T| \quad (12)$$

where $|T|$ is the number of terminal nodes

- ▶ The parameter α controls the trade-off between accuracy and conciseness.
- ▶ For each α there is a unique subtree T_α that minimises C_α .
- ▶ To find T_α , we use *weakest link pruning*: we collapse the non-terminal node with the smallest error contribution iteratively, until we obtain a single node tree
- ▶ This gives a (finite) sequence of subtrees, and one can show this sequence must contain T_α

Interpreting a decision tree

- ▶ Once a model has been trained, it can be beneficial to use it to obtain an interpretable solution to the classification or regression problem.
- ▶ A decision tree can be seen as a collection of rules
 - ▶ Each leaf node (or partition) corresponds to *conjunctive clause*, where each non-leaf node between itself and the root specifies a literal, eg:

$$(x_a \leq \tau_a) \wedge (x_b > \tau_b) \wedge \dots (x_q \leq \tau_q) \rightarrow \lambda$$

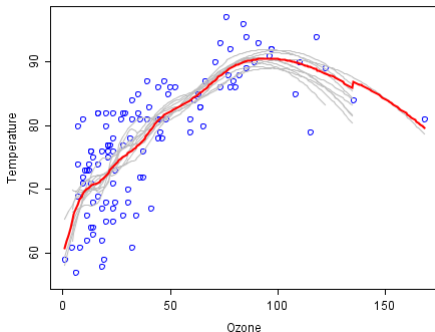
- ▶ A tree is simply a list of such clauses.

- ▶ Decision trees (DTs) are conceptually simple, but at the same time practically powerful predictors
- ▶ **Q:** How can we improve the performance of DTs?
- ▶ **A:** We can reduce the predictive error by
 - 1) reducing the bias or
 - 2) reducing the variance
- ▶ Variance reduction: bootstrap aggregation **bagging**
- ▶ Bias reduction: **boosting**

Bagging: Bootstrap Aggregation

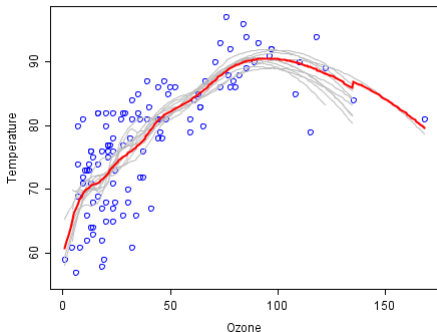
- ▶ Consider a regression problem, with a training set $Z = \{(x_n, y_n)\}_n^N$
- ▶ Let's generate B new training sets Z_b by **randomly sampling** $N' \leq N$ examples from Z (with replacement).
- ▶ We then train B models $\hat{f}_1, \dots, \hat{f}_b$ from the bootstrap samples Z_b , the bagging estimate is given by:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x) \quad (13)$$



Bagging: Bootstrap Aggregation

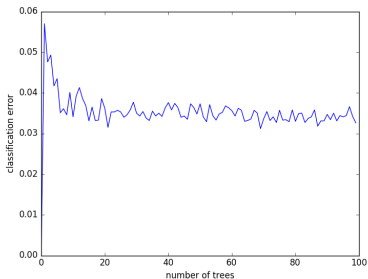
- ▶ Each model is a different realisation of the learning algorithm, and therefore by averaging their response we **reduce overall prediction variance**.
- ▶ the bagging ratio is given by $\mu = \frac{N'}{N}$
- ▶ even with $N' = N$, some samples will be duplicated and some will be missing in each D_i , and therefore the trained models will differ.



Regression example on relationship between ozone and temperature.

Source <https://en.wikipedia.org/wiki/File:Ozone.png>

Effect of the number of trees



- ▶ Effect of the number of trees on the performance of a simple binary classification task: The features are the x and y coordinates of the points, and the labels are denoted by the green and blue colour.
- ▶ Note that performance is significantly improved by the first 10 trees. More trees may be required in more challenging problems.
- ▶ Note that there is no problem of overfitting by increasing the number of trees! Only increased runtime.

- ▶ Trees are a good choice of a predictor for bagging as they have high variance but low bias (if deep enough).
- ▶ Bagging ensure that all trees are identically distributed, hence the expectation of an average over B is the same as the expectation of any individual tree.
- ▶ Hence, bias is the same for all tree, and only possible improvement is by variance reduction.

- ▶ Averaging over B i.i.d. variables, each with variance σ^2 , has a variance of

$$\frac{\sigma^2}{B}$$

- ▶ If variables are correlated by ρ (not independent) then the variance is:

$$\frac{1 - \rho}{B} \sigma^2 + \rho \sigma^2$$

- ▶ As we increase B then the first term will vanish
- ▶ However the second term is not affected by B but only by ρ
- ▶ $\Rightarrow$ bagging is effective only if we work with uncorrelated predictors!
- ▶ Working with the same model class on a finite dataset will eventually lead to correlated predictors

- ▶ **Q:** How to reduce tree correlation?
- ▶ **A:** Inject randomness in the learning process!

- ▶ Different types of random forests can be achieved by changing the source of randomization
 - ① on training set: bootstrapped replicates
 - ② on outputs: flipped for classification, with added Gaussian noise for regression
 - ③ on split: select randomly one from S best splits on all features
 - ④ on feature selection: select m features at random on each node and then find the best split
 - ⑤ on subspace: select m features at random for the whole decision tree
 - ⑥ on all: on each node select two instances of opposite class at random, select a feature at random, select a split point at random

- ▶ Before each split, select only $D' \leq D$ features at random, as candidates for splitting.
- ▶ Recommended values of D' vary in the literature:
 - ▶ $D' = \log_2(D + 1)$ Breiman (2001)
 - ▶ $D' = \sqrt{D}$ for classification.
 - ▶ $D' = \frac{D}{3}$ for regression.
 - ▶ or even $D' = 1$ for maximal randomisation...

Variable Importance (VI)

Given an induced RF, one can rank the importance of each feature according to:

- ➊ **Gini Score:** at each split, the decrease in Gini impurity for the variable chosen for splitting is recorded; the importance of a variable is *its average Gini impurity decrease* (averaged over all the trees in the forest)
- ➋ **OOB Accuracy:** the importance is defined as the *mean decrease in accuracy* averaged over the bootstrapped *validation* replicates
 - ▶ Given that bootstrapped replicates are used to induce each decision tree, one can use the remaining 30% of data called **out-of-bag** (OOB), as validation set (no need to run an external cross-validation)
 - ▶ We call it the *Permutation Variable Importance Estimate*

Permutation VI Estimate

- ▶ For each bootstrapped replicate
- ▶ Identify the OOB instances
- ▶ Predict the class membership and cumulate the correct prediction counts
- ▶ For each feature $j \in \{1, 2, \dots, p\}$
 - ▶ permute the values of X_j in the OOB sample
 - ▶ use the learned decision tree to make predictions
 - ▶ cumulate the correct prediction counts
- ▶ the importance for variable j is the average difference in accuracy on the OOB between the original setting vs. the permuted case

- ▶ The permutation VI can be used in a backward elimination loop for increased reliability:
 - ▶ Compute the importance of each feature on the full dataset
 - ▶ Remove a fixed percentage of the low scoring features (say 10-20%)
 - ▶ Iterate the training and VI estimate until only one predictor is left
*This procedure allows to find a **nested** set of variables and as a consequence a more robust variable **ranking***
 - ▶ ...or, iterate until the out-of-bag error is within one standard deviation from the minimum out-of-bag error estimate

Rational for the Permutation VI Estimate:

- ▶ When we randomly permute a predictor variable, its original association with the target is lost
- ▶ We then use the permuted variable, together with the remaining non-permuted predictor variables, to predict the target
- ▶ In this case, the prediction accuracy is likely to decrease substantially *if the original variable was associated with the response*

*A gambler, frustrated by persistent horse-racing losses and envious of his friends' winnings, decides to allow a group of his fellow gamblers to make bets on his behalf. He decides he will wager a fixed sum of money in every race, but that **he will apportion his money among his friends based on how well they are doing**. Certainly, if he knew psychically ahead of time which of his friends would win the most, he would naturally have that friend handle all his wagers. Lacking such clairvoyance, however, he attempts to allocate each race's wager in such a way that his total winnings for the season will be reasonably close to what he would have won had he bet everything with the luckiest of his friends.*

Quote from Yoav Freund and Robert E. Schapire. "A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting"
Journal of Computer and System Sciences, 55(1):119-139 (1997)

Weak classifiers

- ▶ Consider a balanced two class problem (i.e. the same number of samples for each class)
- ▶ with the instances denoted by $x \in \mathcal{X}$ and with an associated class code $y(x) \in \{-1, +1\}$.
- ▶ A classifier is a mapping: $\hat{f} : \mathcal{X} \rightarrow \{-1, +1\}$
- ▶ The error rate on a training set $X \subset \mathcal{X}$ of size N is

$$\varepsilon(\hat{f}) = \frac{1}{N} \sum_{x \in X} \mathbb{I} [\hat{f}(x) \neq y(x)] \quad (14)$$

- ▶ A **weak classifier** is a predictor slightly better than chance

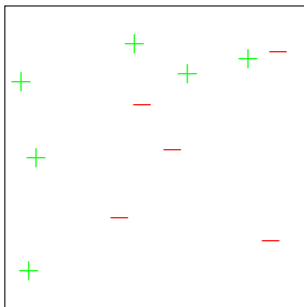
$$\varepsilon(\hat{f}) < 0.5$$

- ▶ Boosting learns a **sequence** of weak predictors $\{\hat{f}_1, \hat{f}_2, \dots, \hat{f}_M\}$ on reweighted versions of the training data.
- ▶ A **strong predictor** can be obtained as a linear combination of weak learners:

$$\hat{f}(x) = \sum_{m=1}^M \alpha_m \hat{f}_m(x) \quad (15)$$

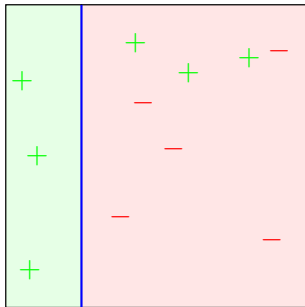
- ▶ The weights $\alpha_1, \dots, \alpha_M$ are calculated to give larger weight to more accurate predictors.

Consider the following binary classification problem:



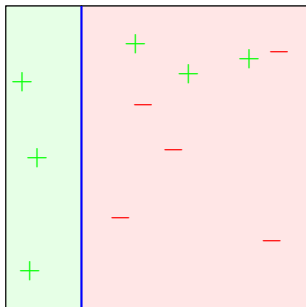
All training example are initialised with the same weight.

Boosting: weak learner



We could train a simple **tree stump**, i.e. a tree with a single split node.

Boosting: weak learner

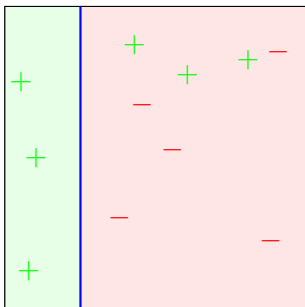


We can calculate the classification error of the stump as:

$$\varepsilon_m = \frac{\sum_{i=1}^N \left(w_i^{(m)} \mathbb{I} \left[y(x_i) \neq \hat{f}_m(x_i) \right] \right)}{\sum_{i=1}^N w_i^{(m)}}$$

where $w_i^{(m)}$ are the weights of training sample i at boosting round m , with $w_i^{(0)} = \frac{1}{N}$ for all i .

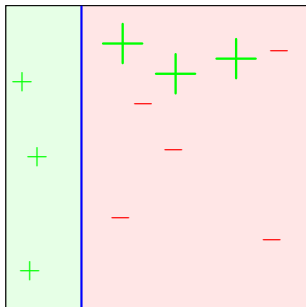
Boosting: weak learner weight



We then estimate the classifier weight α_m from the classification error:

$$\alpha_m = \frac{1}{2} \log \left(\frac{1 - \varepsilon_m}{\varepsilon_m} \right)$$

Boosting: data re-weighting

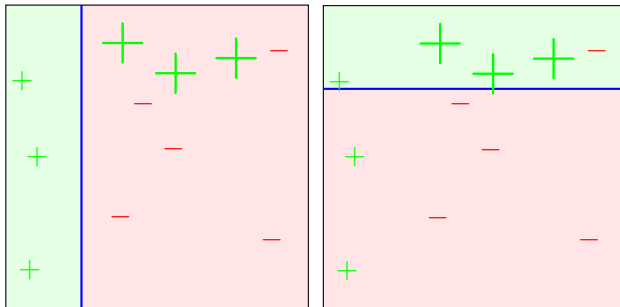


We then increase the weight of misclassified examples:

$$w_i^{(m+1)} \leftarrow \frac{w_i^{(m)}}{Z_m} e^{\alpha_m \mathbb{I}[\hat{f}_m(x_i) \neq y(x_i)]}$$

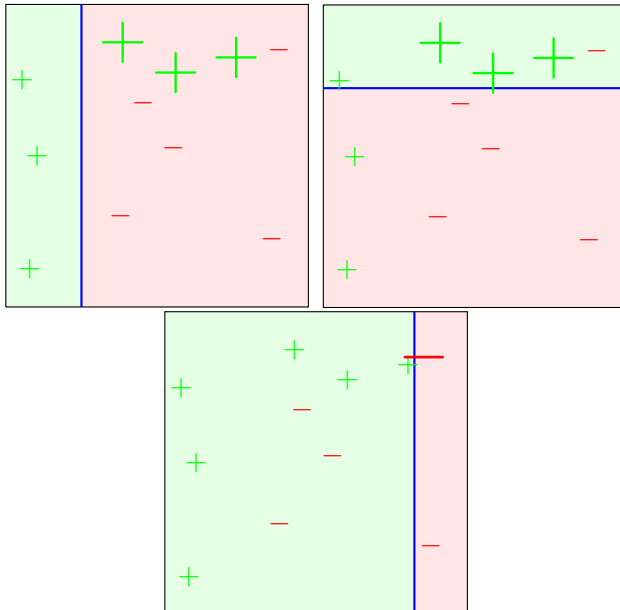
where Z_m is chosen such that $\sum_{i=1}^N w_i^{(m+1)} = 1$.

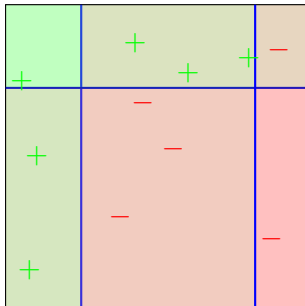
Boosting



And we retrain another weak learner.

Boosting





The linear combination of all weak learners form a **strong learner**:

$$\mathcal{F}(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m \hat{f}_m(x) \right)$$

AdaBoost.M1 (Freund & Schapire, 1997)

- 1: Initialize weights $w = (\frac{1}{N}, \dots, \frac{1}{N})$.
- 2: **for** $m = 1$ to M **do**
- 3: train a weak classifier $\hat{f}_m$ from weights $w^{(m)}$.
- 4: Compute the weighted error:

$$\varepsilon_m = \frac{\sum_{i=1}^N \left(w_i^{(m)} \mathbb{I} \left[y(x_i) \neq \hat{f}_m(x_i) \right] \right)}{\sum_{i=1}^N w_i^{(m)}}$$

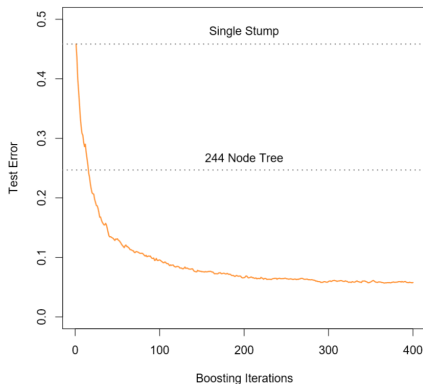
- 5: compute classifier weight

$$\alpha_m = \frac{1}{2} \log \left(\frac{1 - \varepsilon_m}{\varepsilon_m} \right)$$

- 6: update weights: $w_i^{(m+1)} \leftarrow w_i^{(m)} \exp \left[\alpha_m \mathbb{I} \left[y(x_i) \neq \hat{f}_m(x_i) \right] \right]$ for $i \in [1, N]$.
- 7: Normalise weights so that $\sum_{i=1}^N w_i^{(m+1)} = 1$.
- 8: **end for**

The features $X_1, \dots, X_{10}$ are standard independent Gaussian, and the deterministic target Y is defined by

$$Y = \begin{cases} 1 & \text{if } \sum_{j=1}^{10} X_j^2 > \text{th} \\ -1 & \text{otherwise} \end{cases}$$



Hastie, Tibshirani, Friedman, *Elements of Statistical Learning*, Fig. 10.2

Boosted tree stumps (depth=1)

- ▶ Adaboost only requires that the error rate of each weak classifier be below 0.5
- ▶ Note: if $\varepsilon > 0.5$, then $\alpha_m < 0$ and we **decrease** the weights of misclassified examples!
- ▶ Safe to assume for binary cases, but less likely for multi-class problems!
- ▶ SAMME (Zhu et al, 2009) extends boosting allowing for weak learners that have accuracy rates: $(1 - \varepsilon) > \frac{1}{K}$ for a problem with K classes.

SAMME (Zhu et al, 2009)

- 1: Initialize weights $w = (\frac{1}{N}, \dots, \frac{1}{N})$.
- 2: **for** $m = 1$ to M **do**
- 3: train a weak classifier $\hat{f}_m$ from weights w .
- 4: Compute the weighted error:

$$\varepsilon_m = \frac{\sum_{i=1}^N \left(w_i^{(m)} \mathbb{I} \left[y(x_i) \neq \hat{f}_m(x_i) \right] \right)}{\sum_{i=1}^N w_i^{(m)}}$$

- 5: compute classifier weight

$$\alpha_m = \frac{1}{2} \log \left(\frac{1 - \varepsilon_m}{\varepsilon_m} \right) + \log(K - 1)$$

- 6: update weights: $w_i^{(m+1)} \leftarrow w_i^{(m)} \exp \left[\alpha_m \mathbb{I} \left[y(x_i) \neq \hat{f}_m(x_i) \right] \right]$ for $i \in [1, N]$.
- 7: Normalise weights so that $\sum_{i=1}^N w_i^{(m+1)} = 1$.
- 8: **end for**

- ▶ SAMME (Zhu et al, 2009) extends boosting allowing for weak learners that have accuracy rates: $(1 - \varepsilon) > \frac{1}{K}$ for a problem with K classes.
- ▶ With SAMME, the final prediction of the collection is given by:

$$\mathcal{F}(x) = \arg \max_k \sum_{m=1}^M \alpha_m \cdot \mathbb{I} \left[\hat{f}_m(x) = k \right]$$

- ▶ Exponential improvement: Freund & Schapire showed that if all the weak hypotheses are slightly better than random, then the training error drops exponentially fast.
- ▶ Boosting can work on any weak learner.
- ▶ As Boosting lowers the bias, it might also lead to overfit: a regularization technique might be needed (e.g. shrinkage as a form of weight decay or early stopping)

Boosting is

- ▶ resistant to overfitting
- ▶ sensitive to outliers: since each weak classifier tries to repair its predecessors' mistakes, the model may end up focussing too much on outliers
- ▶ not parallelizable, since each estimator is built on the basis of its predecessors

Decision Trees

- ▶ Simple algorithm, handles classification, multiclass, and regression.
- ▶ main parameter is max depth/min sample number.
- ▶ High variance, low bias predictor.

Bagging

- ▶ learn a collection of predictors from random subset of training data.
- ▶ reduce prediction variance.
- ▶ random forests yield very high prediction accuracy.

Boosting

- ▶ linear combination of weak predictors
- ▶ learnt in sequence by reweighting misclassified training samples.

- ▶ Freund, Y. & Schapire, R. (1997), 'A decision theoretic generalization of on-line learning and an application to boosting', *Journal of Computer and System Sciences* **55**(1), 119-139.
- ▶ Zhu, J., Zou, H., Rosset, S. & Hastie, T. (2009), 'Multi-class AdaBoost', *Statistics and Its Interface* **2**(3), 349-360, 2009.
- ▶ Friedman, J., Hastie, T. & Tibshirani, R. (2000), 'Additive logistic regression: a statistical view of boosting', *Annals of Statistics* **28**, 337-407.
- ▶ Breiman, L. (2001), 'Random Forests', *Machine Learning* **45** (1): 5-32.